EF101-P11: Computer Hardware Inventory and Maintenance System

Course: EF-101
Project ID: EF101-P11
Submission: Week 13, Monday, 9 November 2026
Group members: Hiba Khan[208] , Sara Khan[213] ,Hareem Ahmed[215]
Instructor: Dr.Faizan

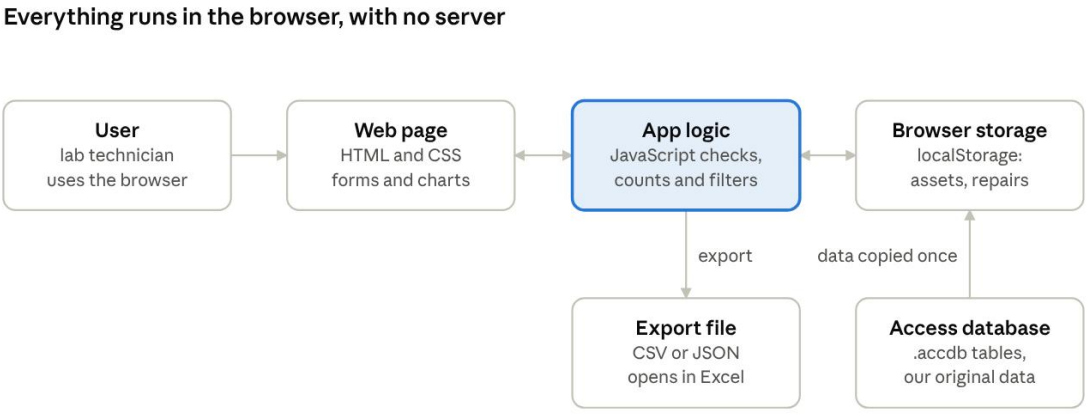
1. Problem statement

In a lab or a small office, nobody really knows what is inside each computer unless someone writes it down. Which PCs are old? Which ones are still under warranty? Which one keeps breaking? Without a record, these questions are answered by guessing, and repairs are repeated because nobody remembers the earlier ones. Our project addresses this by keeping the hardware details and the repair history of every computer in one system.

2. Objectives

- Store the hardware details of every computer (CPU, RAM, storage, GPU, OS, IP address, location, purchase date, warranty and status) in one database.
- Record every fault with its date, description, action taken, technician and cost, and show the history of each asset.
- Let a user add, edit, search, filter and retire assets.
- Show a dashboard and at least one hardware chart, and find the most frequent failure.
- Export the inventory and maintenance data so it can be used in Excel or Access.

3. System architecture



architecture diagram · 5 components, no server

The system has five parts, and it runs completely in the browser. There is no server, and it does not need internet access. The diagram above shows how data moves between the parts.

- **User**: a lab technician who uses the web page in a browser.
- **Web page (HTML and CSS)**: shows the dashboard, the tables, the charts and the forms, and passes every click to the app logic.
- **App logic (JavaScript):** checks the input, counts and filters the data, and works out the dashboard numbers. It is the part that actually does the calculations.
- **Browser storage (local Storage):** keeps the assets and the maintenance records, so changes are still there after the page is closed.
- **Access database:** our original database with the tables, the relationship and the queries. It is not connected live to the web page. The data was copied from it into the web page once, at the start.
- **Export file:** a CSV or JSON file saved by the app logic, which opens in Excel or can be imported into Access.

When a button is pressed, the web page hands the action to the app logic. The logic checks the input, reads or changes the data in browser storage, and sends the result back to the page, which draws itself again.

4. Technology selection

Tool	Used for	Why we chose it
Microsoft Access	The original database: tables, relationship and queries	Easy for beginners, supports relationships and queries, and is common in labs
HTML and CSS	The web page and how it looks	Opens in any browser with nothing to install
JavaScript	Dashboard calculations, filters, forms and input checks	It runs inside the browser, so no server is needed
localStorage	Saving changes	Built into the browser, so no separate database server is needed
CSV and JSON	Exporting data	Opens in Excel and can be imported into Access
Claude (AI assistant)	Help with the web page and the report	See section 10

We did not use any external libraries, APIs or cloud services.

5. Data design

The database has two main tables. One computer can have many repairs over time, so the Assets table is linked to the Maintenance table by the asset ID.

The Assets table has one row per computer (30 rows). Its fields are AssetID, AssetTag, Manufacturer, Model, CPU, RAM_GB, Storage_GB, Storage Type, GPU, Operating System, IP Address, Location, Purchase Date, Warranty Status and Operational Status. In the web page we added one more field, Active, which is how an asset is retired.

The Maintenance table has one row per fault (15 rows). Its fields are Maintenance ID, Asset ID, Maintenance Date, Fault Category, Fault Description, Action Taken, Technician and Cost.

We also made queries in Access that join the two tables. They show the full history of each asset, the work done by each technician and the total maintenance cost.

Before building the dashboard we cleaned the data, because spelling mistakes would have given wrong counts. For example, a status written two different ways would be counted as two different statuses. We fixed spellings, made locations and brand names consistent, wrote the model names in one format, and stored cost as a number. One fault had "Faulty" as its category, which is not really a category, so we changed it to "Hardware". A retired asset is not deleted. It is only marked inactive, so its repair history stays in the data.

Why we track each hardware field

The asset ID and asset tag give each computer its own identity, so a repair or warranty claim always points to one machine. The manufacturer and model matter because spare parts, drivers and warranty support depend on them.

The CPU family (Core i3, i5, i7 or Ryzen) tells us roughly how powerful and how old a computer is. RAM and storage are often mixed up, but they are different things. RAM is the short-term memory used while programs are running, and it is emptied when the power goes off. Storage keeps our files permanently. A slow computer may need more RAM, while a computer that is full needs more storage.

SSDs and HDDs are both storage devices, but an SSD has no moving parts, so it is faster and handles bumps better. An HDD has a spinning disk, costs less per GB and is more likely to fail, so an organization wants to know which machines still use one. In the same way, the CPU does the general work of the computer, while the GPU handles graphics. Integrated graphics are fine for office work, but a graphics or AI lab needs a proper GPU.

The operating system matters for security updates and licences. The asset ID and the IP address are also different. The asset ID is permanent and chosen by the organization, but the IP address belongs to the network and can change, so it only tells us where the device is on the network right now.

The location tells a technician where to go. The purchase date and warranty status decide whether a repair is free or has to be paid for, and the purchase date also shows how old the machine is. The operational status (operational, faulty or under maintenance) shows at a glance how many computers can actually be used.

Maintenance history is probably the most useful part. It shows which computers fail again and again, which parts fail most, how much money goes into repairs and how busy each technician is. If one machine keeps coming back with problems, it may be cheaper to replace it than to keep repairing it.

6. Implementation and method

We worked in this order. First we designed the two tables in Access, entered the 30 assets and 15 repairs, and linked the tables. Then we cleaned the data and wrote the queries. After that, the data was put into the web page, and the features were built on top of it.

The dashboard counts the assets by status and warranty, using only active assets. It lists the assets that need attention (faulty or under maintenance), shows the total repair cost, and draws bar charts for RAM size, SSD versus HDD, CPU family, location, fault category and operating system.

On the Assets page the user can add a new computer or edit an old one. The form refuses an empty required field, a repeated asset tag, a wrong IP address and negative numbers. The user can search by any word and filter by CPU, RAM, storage type, location, status and warranty. Retiring an asset removes it from the dashboard counts but keeps its history.

On the Maintenance page the user can log a new fault. The History button next to each asset shows only that computer's repairs and their total cost. For failure analysis, the app counts the fault categories and the repairs per asset and picks the highest. Hardware is the most common category, and PC-001 and PC-003 are tied for the most repairs with two each.

7. Testing

What we tried	What happened
Added an asset with a tag that already exists	It was rejected with a message
Added an asset with a wrong IP address	It was rejected with a message
Retired an asset	It left the dashboard counts, history stayed
Filtered by RAM and location	Only matching assets were shown
Logged a fault for an asset	It appeared in that asset's history and in the Maintenance page
Exported assets to CSV	A CSV file was saved

One test failed during development. The first version of the web page opened completely blank. The reason was a variable in the code whose name clashed with a name the browser already uses. We found it by running the page in a test, renamed the variable, and then all the pages worked.

8. Results

From the 30 assets and 15 repairs, the dashboard shows how many computers are working, faulty or out of warranty, and which ones need attention now. Hardware faults are the most common, so an organization like this should keep common spare parts such as RAM and power supplies. PC-001 and PC-003 had repeated problems, so they should be checked first when deciding what to replace. The charts also show how RAM sizes, storage types and CPUs are spread across the lab, which helps when planning upgrades.

9. Limitations

The prototype has some limits. Warranty status is typed in by hand, so it does not change by itself. It would be better to store the warranty end date and let the system work out the status. Changes are saved only in the browser that was used, so they are not shared between computers and are not written back to the Access file. There is no server, no login and no backup, and anyone who opens the file can change the data. In a real product we would use a shared database with user accounts.

If we had more time, we would add QR or barcode labels for assets, automatic import of PC specifications, a spare-parts list, maintenance reminders and an age or depreciation dashboard.

10. AI, libraries and external resources

We used an AI assistant, Claude (made by Anthropic), during this project, with our instructor's permission. The database itself was made by us in Microsoft Access: the tables, the 30 asset records, the maintenance records, the relationship between the tables and the queries.

Claude helped us in these ways:

- It built the web page on top of our data, with the dashboard, the forms for adding and editing assets, the search and filters, the charts and the export.
- It helped us clean the data, for example spelling mistakes and the same value written in different ways.
- It pointed out problems in our database, such as misspelled field names and a duplicate table.
- It helped us write the first draft of this report and the explanations of the hardware fields.

We did not use any external libraries, APIs or downloaded data. All 30 assets and 15 repairs are sample data created for this project. [Edit this section if your own contribution was different.]

11. Individual contributions

Member	Contribution
[Name 1]	[for example: designed the Access tables and entered the asset data]
[Name 2]	[for example: maintenance records, queries and data cleaning]
[Name 3]	[for example: testing, screenshots and the report]

12. Conclusion

This project lets an organization record its computers, track faults and repairs, and use that data to make decisions. Along the way we practised database design, cleaning data and building a dashboard, and we learned why each hardware detail matters. We believe it meets the mandatory requirements of EF101-P11.